

telemetryDisplay

Application web de visualisation de télémétrie pour circuit, avec import de fichiers `.mat`, dashboard de graphes, track map et émulation de math channels via des blocs de software.

Structure du projet

- `backend/` : API FastAPI et logique d'import des données
- `frontend/` : interface React + Vite
- `data/` : jeux de données d'exemple et cartes de piste
- `docs/` : conventions de format, dont le format MAT

Prérequis

- **Python 3.8+** pour le backend
- **Node.js** pour le frontend
- Un navigateur moderne (**Chrome** de préférence)

Pour vérifier que les prérequis sont validés vous pouvez taper les commandes suivantes dans un terminal bash:

- `python --version` et vous verrez la version de votre Python apparaître.
- `npm --version` et vous verrez également la version de votre Node.js apparaître, ou une fenêtre s'ouvrir rapidement. Si l'installation n'est pas bonne, npm ne sera pas reconnue en tant commande interne.

Pour installer Node.js, rendez vous sur <https://nodejs.org/en/download/current> et téléchargez le Windows Installer. Suivez les étapes d'installation.

Installation

Git

Vous devez clone le git dans le dossier de votre choix. Pour cela: `git clone https://github.com/camillegervais/telemetryDisplay.git`.

Une fois le git installé, vous pouvez ouvrir un terminal Git Bash dans ce dossier et faire les opérations suivantes:

```
git fetch
git checkout prod
git pull
```

Le repo est maintenant à jour et les dépendances sont prêtes à être installées.

Python

Installation Python

Si vous ne satisfaisiez pas les prérequis sur la version de Python, vous pouvez suivre les étapes suivantes:

1. Rendez vous dans `telemetryDisplay/resources` via l'explorateur de fichier.
2. Lancer `python-3.14.5-amd64.exe` avec les privilèges d'administrateur et suivez les étapes de l'installation.

Après l'installation, vous pouvez vérifier la version de python comme mentionné plus tôt.

Setup backend

Dans le cas où votre Python est fonctionnel, vous pouvez:

Ouvrez un git bash dans `telemetryDisplay` et tapez les commandes suivantes:

```
python -m venv .venv
source .venv/Scripts/activate
python -m pip install -r requirements.txt
```

Node

Dans le cas où votre installation de Node est fonctionnelle, vous pouvez:

Ouvrez un git bash dans `telemetryDisplay` et tapez les commandes suivantes:

```
npm install
cd frontend
npm install
cd ..
```

/!\ Remarque installation Python

L'installation de Python 3.14.5 faite pour cette application peut interférer avec vos autres applications. Si c'est le cas, vous pouvez désinstaller Python 3.14 depuis le menu **Démarrer** comme une application normale. En effet, le **venv** suffit à faire fonctionner l'application vous n'avez plus besoin de l'application globale.

Démarrage

Pour une utilisation quotidienne, préférez le **démarrage complet depuis la racine**.

Démarrage complet depuis la racine

Depuis la racine du dépôt, vous pouvez utiliser le script de dev configuré pour lancer frontend et backend ensemble appelé: `Telemetry Display.sh`. Ce mode suppose que les dépendances sont déjà installées.

Afin de simplifier l'utilisation, vous pouvez créer un raccourci de ce script sur le bureau.

D'autres façons de démarrer l'application sont listées à la fin de ce document et sont réservées à des usages plus avancés de l'application

Prise en compte des mise à jour du code

Lors du développement de l'application, des mises à jour régulières seront disponibles avec des résolutions de bugs ou des nouvelles fonctionnalités, voici la démarche pour les récupérer:

1. Aller dans le dossier où est le code source de l'application (nommé **telemetryDisplay**) et assurez vous qu'aucune instance de l'application n'est ouverte.
2. Clic droit dans la fenêtre et faites **Open Git Bash here**.
3. Assurez vous d'avoir une connexion internet.
4. Tapez dans le terminal ouvert les commandes suivantes, {nom de la branche} est indiquée dans le message de push:

```
git fetch
git checkout {nom de la branche}
git pull
```

5. Relancez l'application avec le script **Telemetry Display.sh**

Si besoin de mettre à jour les bibliothèques Python, depuis **telemetryDisplay**:

```
source .venv/Scripts/activate
python -m pip install -r requirements.txt
deactivate
```

Si besoin de mettre à jours les dépendances Node, depuis **telemetryDisplay**:

```
npm install
cd frontend
npm install
cd ..
```

Utilisation basique

1. Importer un dataset via le bouton **Importer des données**, plusieurs choix s'offrent à vous: sélectionner un fichier .mat, trouver un tour sur via TelDataX ou réimporter des datasets récents.
2. Consulter les signaux disponibles dans la liste de gauche.
3. Ajouter des math channels depuis le même panneau pour créer des signaux dérivés.
4. Ajouter des cartos et modifier les gains/offsets en fonctions de vos besoins.
5. Ouvrir le dashboard pour afficher les graphes et la track map.

Fonctions utiles de l'interface:

- Les blocs de soft sont des suites d'opérations s'appliquant dans l'ordre pour recréer des fonctionnalités du software de la voiture.
- Les blocs de soft sont conservés au rechargement de la page.

- Le mode **Graph Only** masque les panneaux latéraux pour se concentrer sur les graphes.
- Le panneau Graphe Perso sert à modifier widgets, tailles et positions.
- L'aide des raccourcis clavier est accessible depuis le bouton **SHORTCUT** en haut à droite.
- L'onglet **Trajectoire** permet de visualiser la trajectoire de la voiture dans le plan quand les signaux **xCar**, **yCar**, **xTrack** et **yTrack** sont présents.
- L'onglet **Tuning Cartos** permet de créer des LUT qui prennent en entrées des canaux existants et qui pourront être utilisés dans des blocs de soft.
- L'onglet **Soft** permet de gérer les différents blocs de soft dans l'application.
- Les canaux peuvent être visualisés de différentes façons: que les valeurs positives, que les valeurs négatives les valeurs correspondants à des phases de freins (**MBrakeR** != 0)

Format des fichiers MAT

Le format attendu est décrit dans [docs/MAT_FORMAT.md](#).

En résumé:

- **sLap** est obligatoire et doit être monotone croissante.
- Chaque signal doit avoir la même longueur que **sLap**.
- Les **NaN** dans le signal sont acceptés, s'ils sont en fin de signal et uniquement en fin de signal on trim les données du dataset (explosion d'une simulation)

Utilisation avec MATLAB / Simulink

Cette application vise à offrir un outil pratique pour analyser des données de télémétrie issues d'une simulation Simulink. Pour générer les données compatibles avec l'application, procédez ainsi:

1. Copier le fichier **exportDataMATLAB.m** dans le dossier de votre projet Simulink.
2. Le configurer comme callback **StopFcn** de la simulation.
3. Ajouter des blocs **To Workspace** sur les signaux que vous voulez analyser dans l'application.

Autres démarrages

Lancer le backend

1. Aller dans **backend/**.
2. Activer **.venv**, depuis **telemetryDisplay**: `source .venv/Scripts/activate`.
3. Lancer l'API: `uvicorn app.main:app --reload --port 8001`

L'API est disponible sur <http://localhost:8001>.

Lancer le frontend

1. Aller dans **frontend/**.
2. Lancer l'interface: `npm run dev`

L'application est disponible sur <http://localhost:5173> et s'attend à trouver l'API sur <http://localhost:8001>.

Démarrage simultané du frontend et du backend

1. Aller dans `telemetryDisplay`.
2. Lancer l'application au complet avec la commande: `npm run dev`

Vous verrez alors se lancer une fenêtre Node où les informations du frontend seront en cyan et celles du backend en vert.